machine learning & knowledge extraction

MDPI

Article

FPGA-Accelerated ESN with Chaos Training for Financial Time Series Prediction

Zeinab A. Hassaan [1], Mohammed H. Yacoub [2] and Lobna A. Said [1,*]

[1] Nanoelectronics Integrated Systems Center (NISC), Nile University, Giza 12588, Egypt; zhssaan@nu.edu.eg
[2] School of Engineering and Applied Sciences, Nile University, Giza 12588, Egypt; m.hassan2127@nu.edu.eg
* Correspondence: l.a.said@ieee.org

Abstract

Improving financial time series forecasting presents challenges because models often struggle to identify diverse fault patterns in unseen data. This issue is critical in fintech, where accurate and reliable forecasting of financial data is essential for effective risk management and informed investment strategies. This work addresses these challenges by initializing the weights and biases of two proposed models, Gated Recurrent Units (GRUs) and the Echo State Network (ESN), with different chaotic sequences to enhance prediction accuracy and capabilities. We compare reservoir computing (RC) and recurrent neural network (RNN) models with and without the integration of chaotic systems, utilizing standard initialization. The models are validated on six different datasets, including the 500 largest publicly traded companies in the US (S&P500), the Irish Stock Exchange Quotient (ISEQ) dataset, the XAU and USD forex pair (XAU/USD), the USD and JPY forex pair with respect to the currency exchange rate (USD/JPY), Chinese daily stock prices, and the top 100 index of UK companies (FTSE 100). The ESN model, combined with the Lorenz system, achieves the lowest error among other models, reinforcing the effectiveness of chaos-trained models for prediction. The proposed ESN model, accelerated by the Kintex-Ultrascale KCU105 FPGA board, achieves a maximum frequency of 83.5 MHz and a power consumption of 0.677 W. The results of the hardware simulation align with MATLAB R2025b fixed-point analysis.

Keywords: reservoir computing; echo state network; recurrent neural network; chaotic systems; hardware accelerator; field programmable gate array

check for updates

Academic Editor: Fabrizio Riguzzi

Received: 12 October 2025
Revised: 13 November 2025
Accepted: 18 November 2025
Published: 3 December 2025

Citation: Hassaan, Z.A.; Yacoub, M.H.; Said, L.A. FPGA-Accelerated ESN with Chaos Training for Financial Time Series Prediction. *Mach. Learn. Knowl. Extr.* **2025**, *7*, 160. https://doi.org/10.3390/make7040160

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Intelligent decision-making in financial market prediction is essential for a stock trading strategy [1]. Time series data is used across various fields of the natural and social sciences, including biologically inspired systems, population dynamics, financial markets, climate studies, and global flow models [2]. Time series data can be classified into linear and nonlinear categories; however, many actual time series exhibit significant nonlinear characteristics [3]. Predicting chaotic and non-periodic dynamic data is a challenging task; however, deep learning models, as part of machine learning, significantly enhance training effectiveness, leading to high performance and accurate results. Stock price prediction has some challenges, such as capturing patterns, dealing with noisy data, and non-stationary properties in which statistical properties vary over time [4]. In volatile financial markets, prediction models rely on time-domain or frequency-domain analysis, which affects the capture of non-stationary and multi-scale dynamics data. Fourier transform (FT) methods

are efficient for analyzing stationary data, but they do not apply to non-stationary financial time series [5]. The deep learning model can be integrated with chaotic systems to enhance the accuracy of time series forecasting [6] by initializing the neural network's weights and biases with a chaotic sequence, thereby improving the generalization capabilities of the prediction model. RNNs are effective at predicting dynamical systems, but they require significant time and resources for training [7,8]. Reservoir computing (RC) models offer an effective method for time series prediction. However, the lack of additional optimization for the output layer leads to significant variance in training outcomes [7]. The performance of RC improves when multiple reservoir layers are used rather than a single one. RC utilizes dynamical systems as nonlinear generalizations to improve learning features and hidden patterns in complex time series [9]. The Echo State Network (ESN), as a type of RC model, is efficient for time series prediction and requires low resource demand for both training and prediction. Hyperparameter optimization reduces variance across initializations [7].

Various RNN and RC models were presented in the literature to improve time series prediction accuracy. A new reservoir computing (RC) approach for mapping input data into the reservoir's state space was proposed in [7]. This approach increased the parallelizability, depth, and predictive capabilities of the neural network model while reducing the dependence on randomness. A novel deep learning method based on the Chen system, called neural basis expansion analysis for interpretable time series forecasting (N-BEATS), was proposed in [6]. This approach improved the performance and efficiency of deep learning models for time-series forecasting. Their experiment was validated on thirteen available time series datasets. A novel approach of the Leverage Convolution LC ARFIMA–GARCH model was proposed in [10] to address the challenges of high-dimensionality, noisy, and non-stationary time series with complex fractal dynamics. The model demonstrated high performance in capturing long-range dependence, whereas the autoregressive approach often fails to capture fragile patterns, producing white noise instead. A modeling strategy, named the vector autoregression (VAR)-based rolling prediction model based on machine learning (ML) techniques, was proposed in [2] for predicting stock prices. A deep learning method combined with a proportional-integral-derivative (PID) error corrector was proposed in [11] for accurate prediction in chaotic time series and the actual pond aquaculture water environment dataset. Three machine learning methods, including RC-ESN, a deep feed-forward artificial neural network (ANN), and RNN–LSTM, were investigated in [12] for predicting short-term evolution and reproducing the long-term statistics of a multiscale spatiotemporal Lorenz 96 system. The RC–ESN method achieved the highest accuracy in forecasting chaotic trajectories. For ESN's reservoir design, an optimization approach was presented in [13] to adapt the reservoir weights based on input data properties, thereby influencing the topology and weights' impact on prediction accuracy. The performance of ESN depends on the reservoir structure, which relies on random weights that are independent of the input data characteristics. A mirrored echo state network (MESN) is constructed in [14] based on a mirrored algorithm to optimize input weights. This methodology mirrors the traditional ESN by exchanging the order of weight determination, thereby creating a mirror symmetry with it. This model was efficient in predicting the Mackey-Glass system (MGS) with large chaotic factors. A sparse compressed deep echo state network (SCDESN) was proposed in [1] with an arithmetic optimization algorithm to optimize the hyperparameters of the SCDESN model. This model was validated on a classic benchmark dataset and two real-world chaotic time series datasets. Grouped vector autoregressive reservoir computing (GVARC) was generated in [15] for time-series forecasting. This model relies on the theory of randomly distributed embedding (RDE) to improve the selection of large-scale parameters in deep RCs. Another method, called Hierarchical Echo State Network with Sparse Learning (HESN-SL), was

proposed in [16] to reduce the approximate collinearity among echo-state information. This model ensured stability when applied to time series forecasting, which satisfies the echo state property. An emerging ML approach, the dendritic neuron model (DNM) with a scale-free differential evolution (SFDE) training algorithm, was applied in [17] to achieve better predictive performance in forecasting financial time series. A novel dual-attention-based sequential auto-encoding architecture, called DAttAE, was proposed in [8] to effectively learn and predict new COVID-19 cases as a chaotic, non-smooth time-series dataset.

Hardware accelerators speed up the complex operations of neural networks compared to general-purpose CPUs [18,19]. Deep learning models comprise numerous complex operations, such as intensive matrix calculations [20], making FPGAs an efficient platform for executing these tasks. The field-programmable gate array (FPGA)-based acceleration of neural networks offers a distinct alternative to Graphics Processing Units (GPUs), Tensor Processing Units (TPUs), and Application-Specific Integrated Circuits (ASICs) [21]. FPGA accelerators are characterized by their programmability, flexibility, improved power efficiency, parallelism, and pipelining in computations [22], as well as scalability [23].

This study explores the potential of deterministic chaotic sequences to outperform traditional random initialization methods in terms of convergence and generalization. The main focus is to improve time-series forecasting by initializing the two proposed models (GRU and ESN) with chaotic sequences and comparing their performance with that of standard initialization. The two proposed models are applied to six different datasets, including the S&P 500 dataset in [6]; ISEQ overall historical data in [10]; and XAU/USD, USD/JPY, Pci-Suntek Tech stock, and FTSE 100 historical data in [2]. The prediction accuracy of the proposed ESN and GRU models is calculated both with and without chaos training. The proposed ESN model, trained using the Lorenz system, achieves the lowest prediction error compared to the GRU model and other models in the literature. The FPGA implementation of the proposed model on Kintex-Ultrascale KCU105 achieves high performance with low power consumption.

This paper is organized as follows: Section 2 describes the datasets employed in this study. Section 3 presents an overview of the chaotic systems and standard initialization, and the mechanism of chaos training. Section 4 outlines the architectures of the proposed models. Section 5 outlines the algorithms for integrating the chaotic system with neural network training and provides a comparison of the proposed models to identify the best prediction model. Section 6 details the hardware implementation of the proposed ESN architecture and its resource utilization results. Finally, Section 7 summarizes the conclusions and suggests future research directions.

2. Overview of the Employed Datasets

2.1. S&P 500

The S&P 500 dataset represents the 500 daily largest publicly traded companies in the US [6]. This data covers about 80% of the US equity market value. It describes the measure of the US large-cap market from 31 March 2015 to 7 April 2025, and it comprises 2522 points, as shown in Figure 1a.

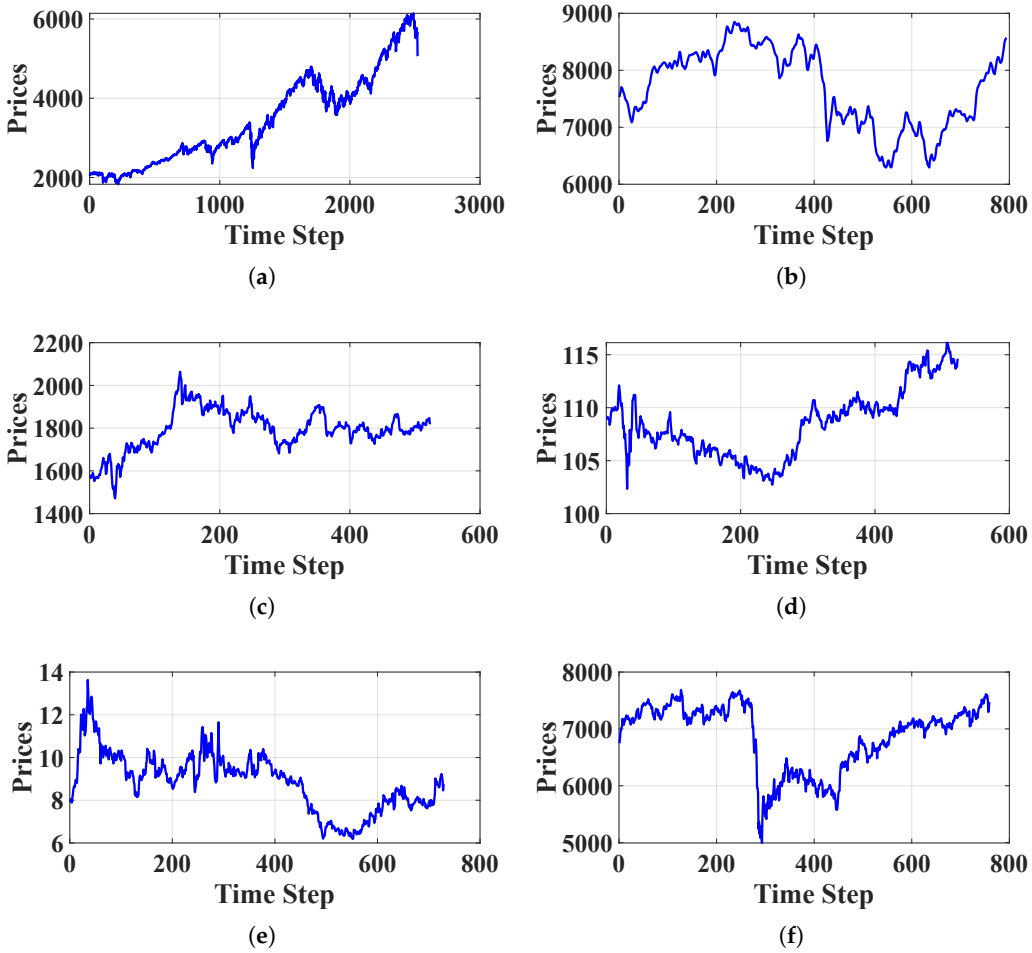

Figure 1. Time series vs. actual prices for the employed datasets: (**a**) S&P 500, (**b**) ISEQ overall historical data, (**c**) XAU USD historical data, (**d**) USD JPY historical data, (**e**) Pci-Suntek Tech stock prices, and (**f**) FTSE 100 historical data.

2.2. ISEQ Overall Historical Data

The Irish Stock Exchange Quotient (ISEQ) dataset represents the daily open price index from 4 January 2021 to 8 March 2023 [10]. It contains 796 points, as shown in Figure 1b. It has chaotic, rapid shocks; non-stationary seasonality; and a negative-skewed distribution. These challenges make forecasting complex.

2.3. XAU/USD Historical Data

XAU/USD contains a forex pair of XAU: ISO 4217 [24] code for gold, where X is the non-currency asset and AU is the atomic symbol for gold, and USD is the US Dollar pricing currency [2]. This data includes the trading day, daily closing price, opening price, highest and lowest price during the trading day, and the daily percentage price change. This dataset represents the gold/US Dollar exchange rate from 29 December 2020 to 20 January 2022, with 524 points. The Intraday timestamps in Figure 1c illustrate the volatility patterns in the dataset.

2.4. USD/JPY Historical Data

The USD/JPY represents the forex pair consisting of the US Dollar as the base currency and the Japanese Yen as the quote currency [2]. This dataset covers the period from 29 December 2020 to 20 January 2022, with 524 points, as shown in Figure 1d. The data

follows standard daily financial market conventions, containing the daily closing, opening, maximum, minimum, and the percentage change in the exchange rate.

2.5. Pci-Suntek Tech Stock Prices (600728.SS)

This dataset includes the Shanghai Stock Exchange (SSE) and provides daily historical prices of Chinese stocks [2]. The symbol 600728 indicates the unique stock code of a Chinese IT company assigned to Pci-Suntek Technology, and $.SS$ is the suffix for the SSE in China's primary stock market. Each data entry contains the date, price, opening price, maximum intraday price, minimum intraday price, volume, and percentage price change. Figure 1e shows the dataset values from 28 January 2019 to 26 January 2022, capturing 730 trading days of market activity.

2.6. FTSE 100 Historical Data (TSCO.L)

This dataset uses the symbol $TSCO.L$, which refers to the UK-based multinational grocery retailer Tesco PLC [2]. The first part, $TSCO$, refers to the Tesco PLC ticker symbol, and the second part $.L$ is the suffix for the London Stock Exchange (LSE). This dataset represents daily historical data for the FTSE 100 index, comprising the market capitalization of the top 100 UK companies. Figure 1f covers the dataset period from 28 January 2019 to 26 January2022, with 760 points. Each entry includes the daily closing price, opening price, high, low, trading volume, and daily percentage change.

3. Overview of the Employed Chaotic Sequences and Standard Initialization in Training Neural Network Models

3.1. The Employed Chaotic Systems

The main characteristics of chaotic systems are determinism and sensitivity to initial conditions [1]. The chaotic initialization mechanism uses a specific base value and then randomly perturbs by a value between 0 and 0.1 [6]. This mechanism generates diverse initial weight configurations, thereby improving prediction accuracy. All chaotic sequences are normalized, and the number of samples for each system equals the total weight needed:

$$No.\,of\ samples = (num_Gates \times hidden_size)(input_size + hidden_size + 1). \quad (1)$$

3.1.1. Standard Chen System

The Chen system is a three-dimensional continuous dynamical system [25]. The chaotic sequence of the Chen system is generated by

$$\frac{dx}{dt} = a(y - x), \quad (2a)$$

$$\frac{dy}{dt} = (c - a)x - xz + cy, \quad (2b)$$

$$\frac{dz}{dt} = xy - bz, \quad (2c)$$

where x, y, and z are the state variables. The parameters a, b, and c have the values 35, 3, and 28, respectively, with time step $dt = 0.005$. The initial conditions are represented by

$$x(0) = y(0) = z(0) = \sqrt{(2)} + 0.1 \times rand(). \quad (3)$$

The three-phase plot of the Chen system is shown in Figure 2a.

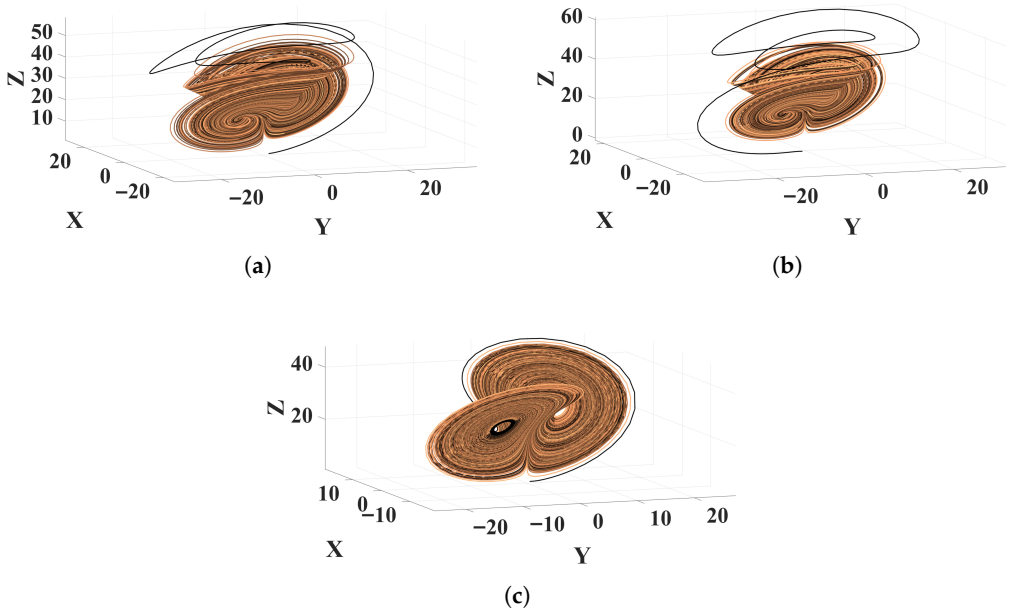

Figure 2. Chaotic system's three phases: (**a**) standard Chen system, (**b**) perturbed Chen oscillator, and (**c**) Lorenz system.

3.1.2. Perturbed Chen Oscillator

The perturbed Chen oscillator (PCO) is a chaotic system derived from the well-known Chen system, a type of continuous-time dynamical system. The system is described as follows:

$$\frac{dx}{dt} = a(y - x), \tag{4a}$$

$$\frac{dy}{dt} = (c - a)x - xz + cy + d, \tag{4b}$$

$$\frac{dz}{dt} = xy - bz, \tag{4c}$$

where d is the perturbation term, and a, b, and c are the oscillator parameters. The three-phase plot of the PCO oscillator is shown in Figure 2b. Chaotic behavior is observed by configuring the PCO parameters, $a = 35$, $b = 3$, $c = 28$, and $d = 5$, with the time step $dt = 0.005$ and the initial conditions:

$$x(0) = y(0) = z(0) = \sqrt{(2)} + 0.1 \times rand(). \tag{5}$$

3.1.3. The Lorenz System

The Lorenz oscillator is a well-known, widely used chaotic system that arises from simplified models of atmospheric convection [26]. The system is defined as follows:

$$\frac{dx}{dt} = \sigma(y - x), \tag{6a}$$

$$\frac{dy}{dt} = x(\rho - z) - y, \tag{6b}$$

$$\frac{dz}{dt} = xy - \beta z, \tag{6c}$$

where σ, ρ, and β are the system parameters governing the oscillator's behavior. The chaotic behavior of the Lorenz oscillator is observed when configuring the system parameters, $\sigma = 10$, $\rho = 28$, and $\beta = \frac{8}{3}$, with time step $\Delta t = 0.005$ and the initial conditions:

$$x(0) = y(0) = z(0) = 0.1 \times randn(). \tag{7}$$

The three-phase plot of the Lorenz system is shown in Figure 2c.

3.1.4. Mackey–Glass Time Series

The Mackey–Glass system is a time-delay differential equation represented by [7,13]:

$$\frac{dx(t)}{dt} = \beta \frac{x(t-\tau)}{1+[x(t-\tau)]^n} - \gamma x(t), \tag{8}$$

where the values of the parameters β, γ, τ, and n are 0.2, 0.1, 17, and 10, respectively. The symbol $x(t-\tau)$ represents the delayed x at time $t-\tau$ with time step $\Delta t = 1.0$. The time series of the Mackey–Glass equation depends on the value of parameter τ, displaying a range of periodic and chaotic dynamics. The system shows non-chaotic or periodic behavior for $\tau < 17$ and chaotic behavior for $\tau \geq 17$, as shown in Figure 3.

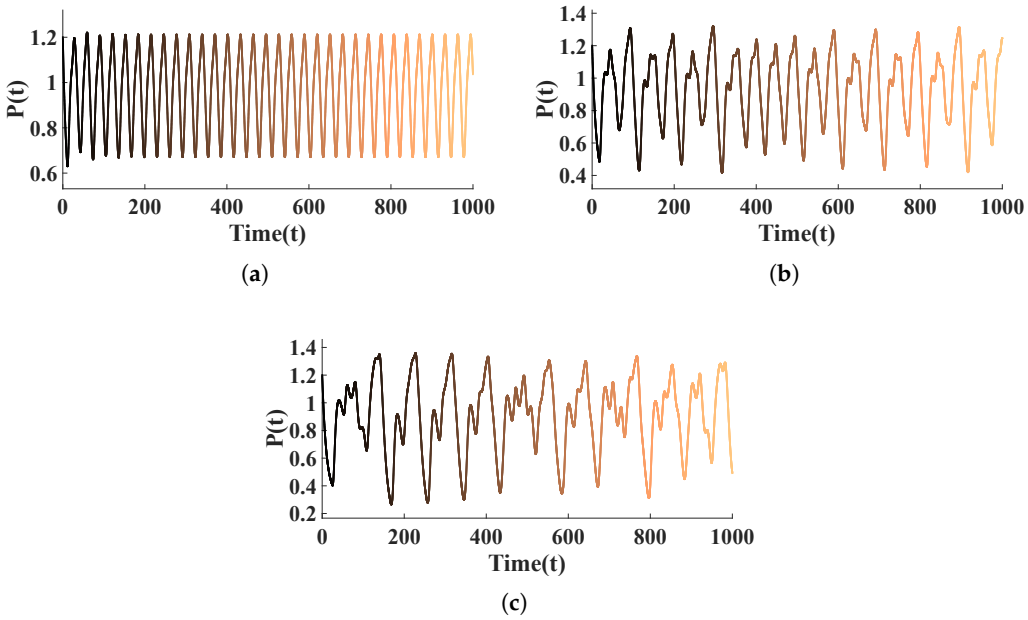

Figure 3. Mackey–Glass equation time series for different values of τ: (**a**) $\tau = 10$, (**b**) $\tau = 17$, and (**c**) $\tau = 25$.

3.2. He and Xavier Standard Initialization

The standard He initialization was proposed by Kaiming He for networks using ReLU activation functions [6]. It addresses the vanishing gradient problem by scaling weights based on the number of neurons. Xavier Glorot proposed the standard Xavier initialization for sigmoid/tanh activations. It balances the variance between input and output dimensions. The initialization of weights with the He and Xavier models is represented by the following:

$$W_{in}^{He} = randn(reservoir_size, input_size) \times \sqrt{\frac{2}{input_size}}, \tag{9a}$$

$$W_{in}^{Xavier} = (rand(reservoir_size, input_size) \times 2 - 1) \times \sqrt{\frac{6}{input_size + reservoir_size}}. \tag{9b}$$

3.3. Chaotic System Distribution

Figure 4 illustrates the histogram analysis for the chaotic and standard initialization using the built-in function '*histogram*' in MATLAB. The standard Chen system in Figure 4a shows a broad distribution of values between −25 and 25. The perturbed Chen oscillator (PCO) shown in Figure 4b widens the distribution from −30 to 30 compared to the standard Chen system. The additional perturbation introduces more extreme values and increases chaotic variability. The Lorenz system in Figure 4c shows a broad distribution from −25 to 25. The Mackey–Glass system in Figure 4d shows a concentrated distribution from 0.8 to 1.6 with a heavy-tailed skew, reflecting its time-delay-driven quasi-periodic chaos. He initialization in Figure 4e shows a narrow Gaussian distribution centered at 0 ranging from −0.5 to 0.5, scaled by $2/n$ for ReLU networks. This ensures stable gradient flow by preventing variance explosion. Xavier initialization in Figure 4f is similar to He but scaled by $1/n$ for tanh/sigmoid activations. The distribution is slightly narrower and optimized for saturating activation functions.

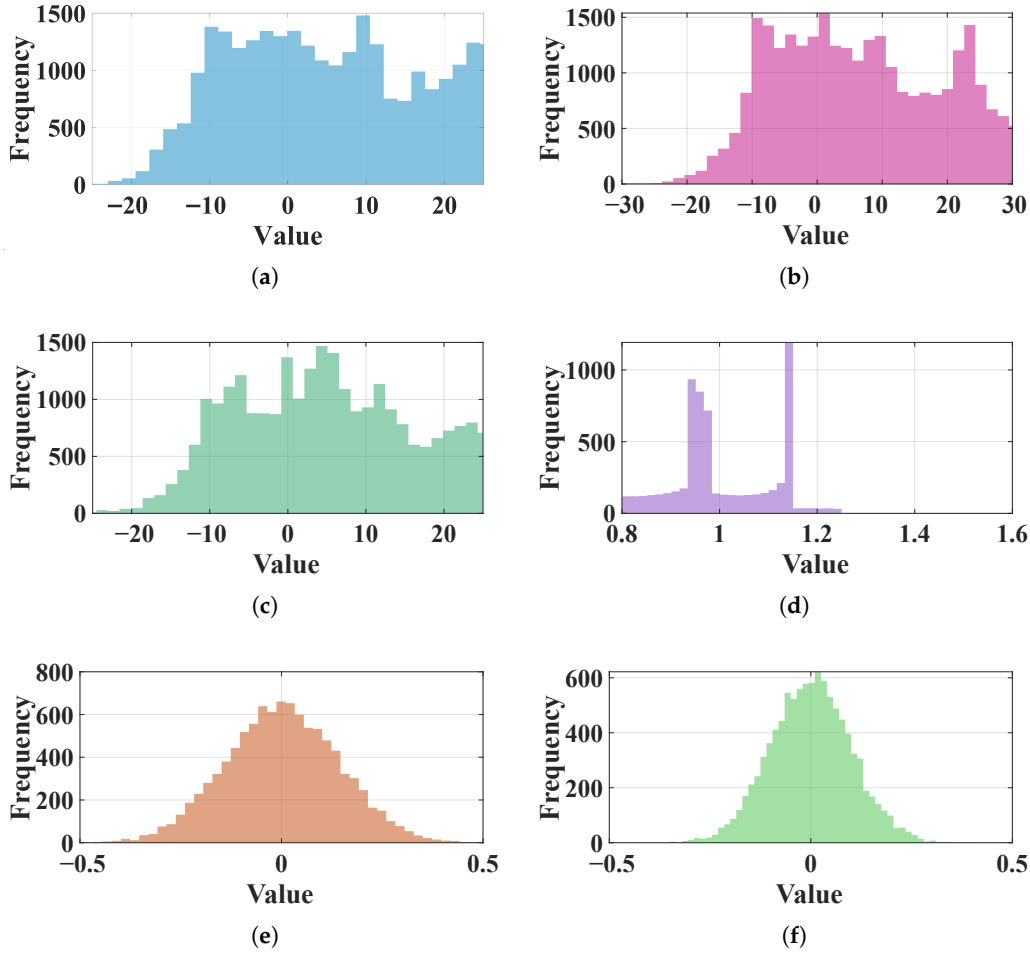

Figure 4. The distribution for the following: (**a**) standard Chen system, (**b**) perturbed Chen oscillator, (**c**) Lorenz system, (**d**) Mackey–Glass equation, (**e**) He initialization, and (**f**) Xavier initialization.

Figure 5 provides a comparative analysis between chaotic and random initialization distributions. Integrating chaotic systems into deep learning models for weight initialization helps prevent vanishing gradients, enabling faster convergence and reduced overfitting [6]. These deep learning models capture long-term dependencies and nonlinear behavior in time-series data due to the fundamental structural and dynamical properties of chaotic systems. Chaotic distribution improves predictive performance compared to standard random initialization.

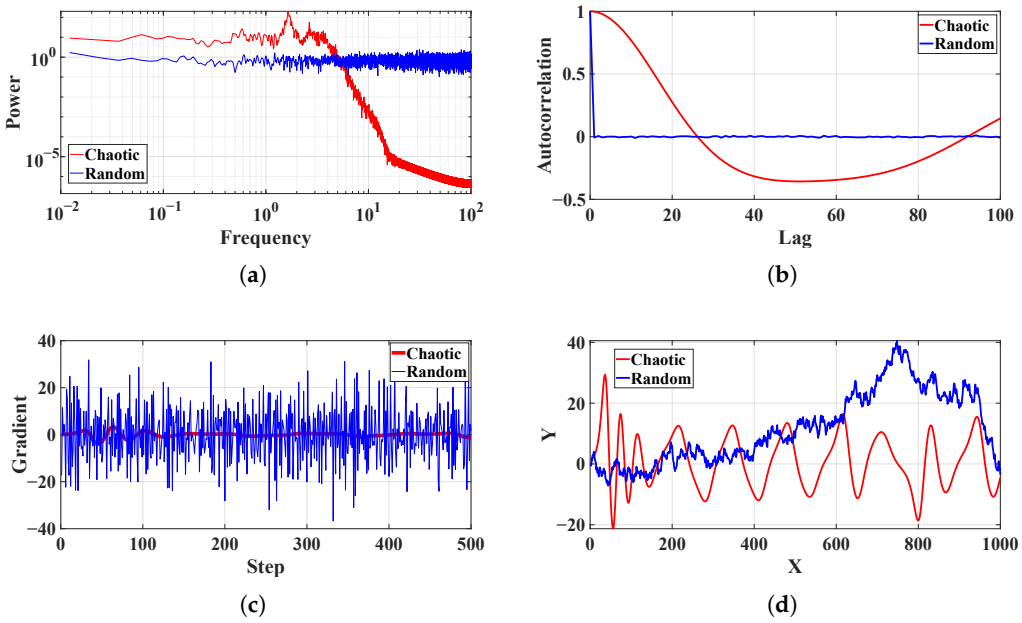

Figure 5. Chaotic vs. random distribution analysis: (**a**) power spectrum, (**b**) autocorrelation comparison, (**c**) gradient patterns, and (**d**) phase space.

4. Recurrent Neural Network and Reservoir Computing Model: General Architectures

4.1. Gated Recurrent Unit Model

The GRU architecture extends conventional RNNs to address the challenge of capturing long-term temporal dependencies [22,25]. The GRU architecture is simpler than the LSTM since it does not include a separate memory cell [27]. GRU consists of two gates: the update gate and the reset gate [28]. The update gate regulates the amount of retained past information, while the reset gate governs the proportion of the previous hidden state to discard. The general architecture of the GRU model is shown in Figure 6a. The internal operations of a GRU cell are defined as:

$$z_{update}(t) = \sigma(W_z x(t) + U_z h(t-1) + b_z), \tag{10a}$$

$$r_{reset}(t) = \sigma(W_r x(t) + U_r h(t-1) + b_r), \tag{10b}$$

$$g_{candidate}(t) = \tanh(W_g x(t) + U_g (r_{reset}(t) \otimes h(t-1)) + b_g), \tag{10c}$$

$$h_{next}(t) = ((1 - z_{update}(t)) \otimes h(t-1)) + (z_{update}(t) \otimes g_{candidate}(t)), \tag{10d}$$

where W_i, U_i, and b_i represent the input weights, recurrent weights, and bias matrices, respectively. The symbol $z_{update}(t)$ refers to the update gate, $r_{reset}(t)$ is the reset gate, $g_{candidate}(t)$ is the candidate hidden state, $h(t-1)$ is the previous hidden state, and $h_{next}(t)$ is the current state. The hidden state at the final time step is used to generate predictions.

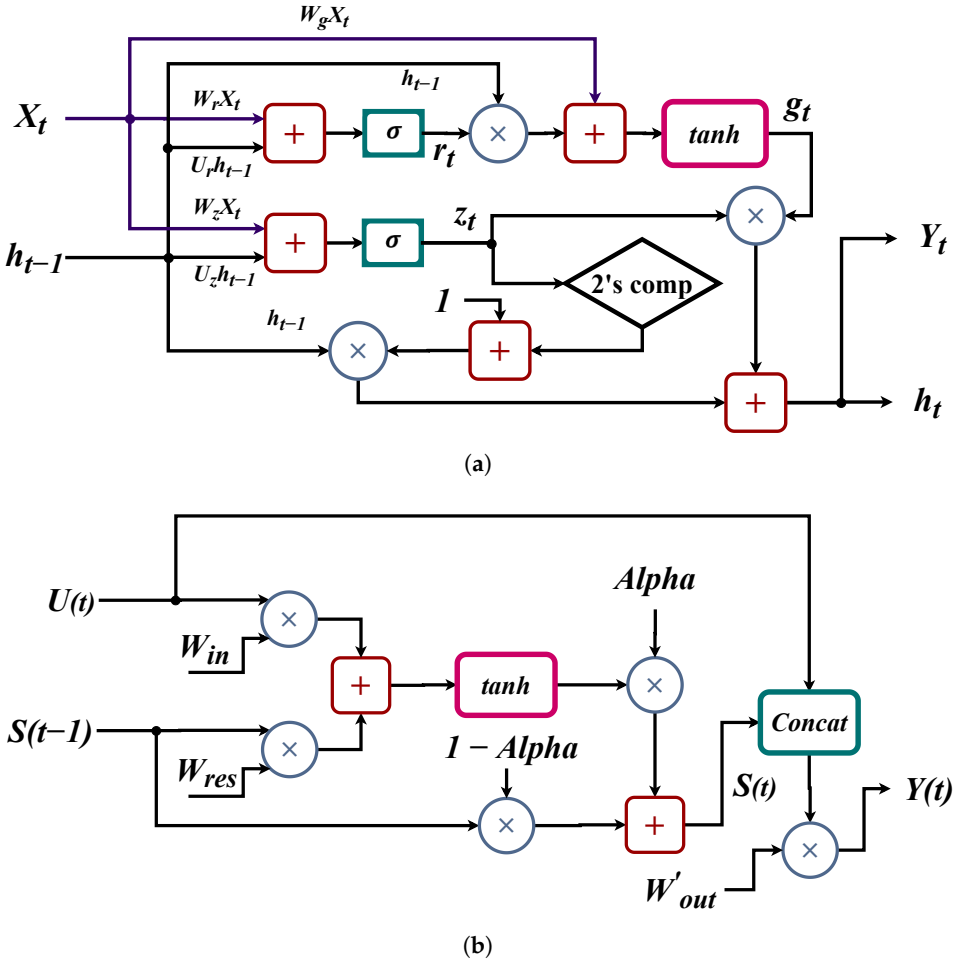

Figure 6. General architectures of: (**a**) the GRU model and (**b**) the ESN model.

4.2. Echo State Network Model

An ESN is a variant of RNNs that consists of an input layer, a dynamic reservoir (hidden layer), and an output layer [18,29]. The hidden layer provides complex nonlinear mappings and encodes the input signals from a low-dimensional input space to a high-dimensional state space. Computation occurs mainly within the reservoir neurons, which reduces training time [30,31]. The ESN architecture is shown in Figure 6b. The main ESN hyperparameters include: spectral radius ρ controls the stability of the reservoir dynamics, input scaling s regulates input magnitude, and sparsity p determines reservoir connectivity. The reservoir state transforms inputs nonlinearly while retaining historical information. The state update is calculated by

$$s^{(t)} = (1-\alpha)s^{(t-1)} + \alpha \tanh\left(W_{\text{in}}u^{(t)} + W_{\text{res}}s^{(t-1)}\right), \tag{11}$$

where $u(t)$ is the input at time t, and α is the leakage rate that balances the previous state s_{t-1} and the new input-driven activation. The nonlinearity *tanh* function ensures that values are between -1 and 1. The input weights W_{in} and the reservoir weights W_{res} are randomly generated. The training phase is separated into two stages. The first stage collects extended states:

$$X = [u(t); s(t)]. \tag{12}$$

The second stage is the ridge regression for the output weights. Only the output weights are learned during training [32]. The output weights are computed by

$$W_{\text{out}} = \left(X^\top X + \lambda I\right)^{-1} X^\top Y_{\text{target}}, \tag{13}$$

where λ is the regularization coefficient that prevents overfitting in ridge regression. The symbol I is the identity matrix of size $\mathbb{R}^{N_r \times N_r}$, where N_r is the number of neurons in the reservoir layer. The target output is Y_{target}. The autoregressive prediction generates next-step predictions using the learned weights. The normalized predicted output is calculated by

$$\hat{y}(t) = W_{\text{out}}^\top [u(t); s(t)]. \tag{14}$$

The state update during prediction is computed by

$$s^{(t)} = (1-\alpha)s^{(t-1)} + \alpha \tanh\left(W_{\text{in}}\hat{y}^{(t-1)} + W_{\text{res}}s^{(t-1)}\right). \tag{15}$$

5. Prediction Models Validation

5.1. Chaotic Initialization Methodology

The GRU model features a fully trainable RNN with a much larger parameter set, while the ESN model has fewer trainable parameters, as shown in Table 1. The architecture of the GRU model consists of two hidden layers, each with 64 neurons. Both models use a sequential forecasting strategy, where multiple independent predictions are generated across the test dataset using a window size of 10 and a forecast horizon of 1. Data preprocessing begins by handling the missing values in the original dataset using linear interpolation. The activation functions are sensitive to their input range; the tanh function outputs values in the range $[-1, 1]$, while the sigmoid outputs values in the range $[0, 1]$. Normalizing the input data ensures that values fall within appropriate ranges for these activation functions, maintaining effective gradient propagation during training. The input features are normalized using min–max scaling to ensure stable and efficient network training. The normalization is computed using the following formula:

$$y^{\text{norm}} = \frac{y - min_y}{max_y - min_y}. \tag{16}$$

Table 1. Quantitative comparison of the key characteristic parameters of the proposed GRU and ESN models.

Model	Key Parameters	Trainable Parameters	Optimization
ESN	Reservoir size = 100 Spectral radius = 0.9 Leakage rate = 0.3 Sparsity = 0.1 Regularization = 1×10^{-4}	Only output weights: 101 parameters	No backpropagation; only linear regression
GRU	Hidden size = 64 Epochs = 150 Batch size = 64 Learning rate = 0.001 Dropout = 0.2	Weights and biases (for two hidden layers) + FC-layer weights: ∼50,000 parameters	Gradient-based with adaptive learning rate

Algorithm 1 demonstrates the chaotic initialization for weights in ESN training. Multiple runs are necessary to ensure the model's prediction accuracy [7]. For chaotic weight initialization, samples are taken from a normalized chaotic time series. The same chaotic sequence is used for all weight matrices in the network.

Algorithm 1 Chaotic Sequence-Initiated ESN

Input: $X_{\text{train}}, Y_{\text{train}}, X_{\text{test}}, Y_{\text{test}}$, Reservoir size: N_r, Spectral radius: ρ, Leakage rate: α, Regularization: λ, Sparsity: s, Training ratio: r_{train}
procedure INITIALIZECHAOTICWEIGHTS($C_{seq}^{norm}, N_r, N_i, s, \rho$)
    $W_{\text{in}} \leftarrow max(min(C_{seq}^{norm}(1 : Nr \times Ni), 1), -1) \times 0.1$
        ▷ Clip to $[-1, 1]$ and scale to $[-0.1, 0.1]$
    $[i, j] \leftarrow find(sprand(N_r, N_i, s))$
        ▷ Indices of sparse matrix with approximately $(s \times N_r \times N_i)$
    nonZeroVals $\leftarrow max(min(C_{seq}^{norm}(1 : length(i)), 1), -1) \times 0.1$
        ▷ Extract non-zero values
    $W_{\text{res}}^{\text{initialization}} \leftarrow \text{sparse}(i, j, \text{nonZeroVals}, N_r, N_r)$
    $\lambda_{\max} \leftarrow \max(|\text{eigs}(W_{\text{res}}^{\text{initialization}})|)$
    $W_{\text{res}} \leftarrow W_{\text{res}}^{\text{initialization}} \times (\rho / \lambda_{\max})$
    return $W_{\text{in}}, W_{\text{res}}$
end procedure
procedure TRAINESN($X_{\text{train}}, Y_{\text{train}}, W_{\text{in}}, W_{\text{res}}, \alpha, \lambda$)
    Initialize reservoir state: $\mathbf{s} \leftarrow \mathbf{0}_{N_r \times 1}$
    for each sequence $\mathbf{u}^{(k)} \in X_{\text{train}}$ **do**
        for $t = 1$ to W **do**
            $\mathbf{s} \leftarrow (1 - \alpha)\mathbf{s} + \alpha \tanh(W_{\text{in}}\mathbf{u}_t + W_{\text{res}}\mathbf{s})$
        end for
        $\mathbf{X}[k, :] \leftarrow [\mathbf{u}_W; \mathbf{s}]^\top$
    end for
    Compute output weights:
    $W_{\text{out}} \leftarrow (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top Y_{\text{train}}$
    return W_{out}
end procedure
procedure MAIN
    for $run = 1$ to 10 **do**
        for each method $m \in$ {Chen, Lorenz, MackeyGlass, PCO, He, Xavier} **do**
            if m is chaotic method **then**
                $C_{seq}^{norm} \leftarrow \text{GenerateChaoticSequence}(m)$
                $W_{\text{in}}, W_{\text{res}} \leftarrow \text{InitializeChaoticWeights}(C_{seq}^{norm}, N_r, N_i, s, \rho)$
            else
                $W_{\text{in}}, W_{\text{res}} \leftarrow \text{StandardInitialization}(m, N_r, N_i, s, \rho)$
            end if
        end for
    end for
end procedure

5.2. Prediction Result Comparison

Figures 7 and 8 illustrate how different chaotic systems influence the prediction performance of the GRU and ESN models across ten independent runs. The ESN model with the Lorenz system achieves the best predictive performance, as shown in Table 2. Performance metrics are computed on the normalized prediction values using the following equations:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i^{\text{norm}} - \hat{y}_i^{\text{norm}})^2, \tag{17a}$$

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i^{\text{norm}} - \hat{y}_i^{\text{norm}}|, \tag{17b}$$

$$\text{MAPE} = \frac{100}{N}\sum_{i=1}^{N}\left|\frac{y_i^{\text{norm}} - \hat{y}_i^{\text{norm}}}{y_i^{\text{norm}}}\right|, \tag{17c}$$

$$\text{Accuracy} = 1 - \text{MAPE}, \tag{17d}$$

where MSE is the mean square error, MAE is the mean absolute error, and MAPE is the mean absolute percentage error.

Table 2. Performance metric comparisons for different datasets with and without chaos training models.

Employed Datasets	Models	Performance Metrics					
		MSE		MAE		MAPE	
		Chaotic	Non-Chaotic	Chaotic	Non-Chaotic	Chaotic	Non-Chaotic
S&P 500	GRU (proposed)	0.000183	0.000129	0.043727	0.009958	5.0875%	1.2055%
	ESN (proposed)	0.000006	0.00659	0.0017	0.0627	0.209923%	2.357%
	N-BEATS in [6]	4.031	5.562	____	____	0.389%	0.468%
ISEQ Historical Data	GRU (proposed)	0.00211053	0.000080316	0.0342342	0.0068244	6.07399%	2.1758%
	ESN (proposed)	0.000034	0.015005	0.00413	0.089523	0.842156%	____
	LC ARFIMA GARCH in [10]	____	603	____	603	____	0.072
XAU/USD Historical Data	GRU (proposed)	0.000568	0.0006516	0.018667	0.02112	3.26004%	3.6862%
	ESN (proposed)	0.0004659	0.015696	0.017021	0.0959	3.0093%	____
	VAR in [2]	____	2.625	____	____	____	____
USD/ JPY Historical Data	GRU (proposed)	0.004955	0.002373	0.057375	0.037068	4.62643%	3.0454%
	ESN (proposed)	0.002201	1.8405	0.03501	1.1495	2.8868	____
	VAR in [2]	____	0.0625	____	____	____	____
Pci-Suntek Tech Stock Price	GRU (proposed)	0.00078	0.000855	0.0199	0.02194	7.67069%	8.1976%
	ESN (proposed)	0.000683	0.017508	0.018295	0.093627	6.93355%	____
	VAR in [2]	____	0.0612	____	____	____	____
FTSE 100 Historical Data	GRU (proposed)	0.000775	0.0005818	0.02130	0.018116	2.58066%	2.2101%
	ESN (proposed)	0.0004787	0.008423	0.015755	0.068239	1.8993%	13.6599%
	VAR in [2]	____	1097	____	____	____	____

The font color is to highlight the best results compared to those in the other models.

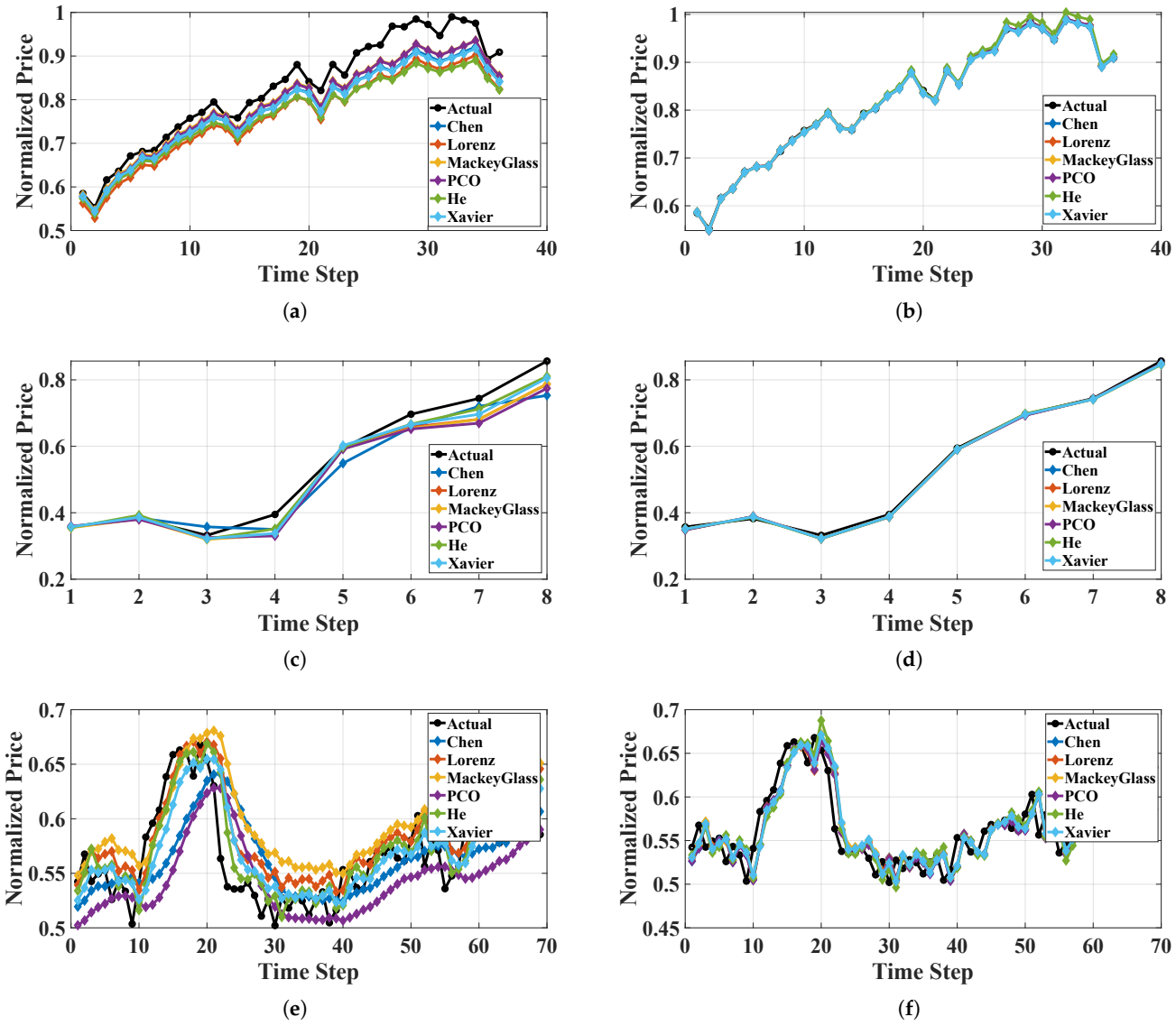

Figure 7. The proposed models' predictions overlaid on the actual data with all chaotic systems: (**a**) GRU for S&P 500 dataset, (**b**) ESN for S&P 500 dataset, (**c**) GRU for ISEQ dataset, (**d**) ESN for ISEQ dataset, (**e**) GRU for XAU/USD dataset, and (**f**) ESN for XAU/USD dataset.

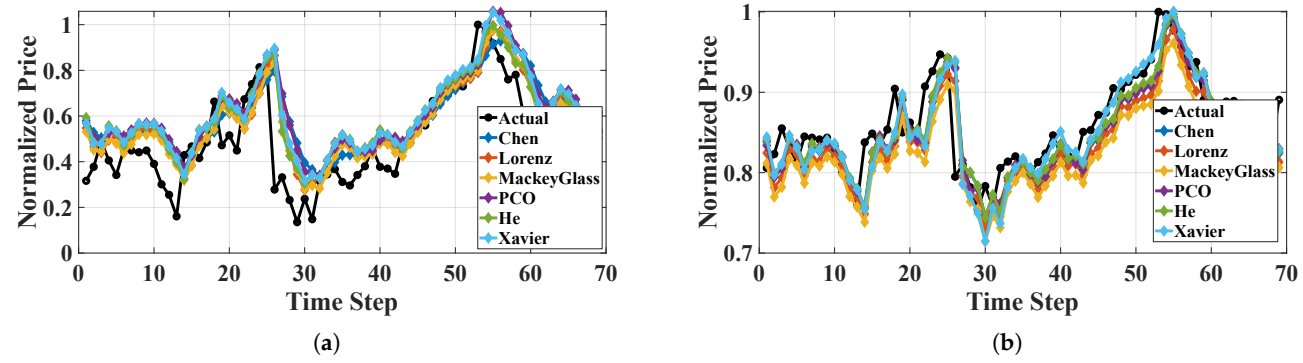

Figure 8. *Cont.*

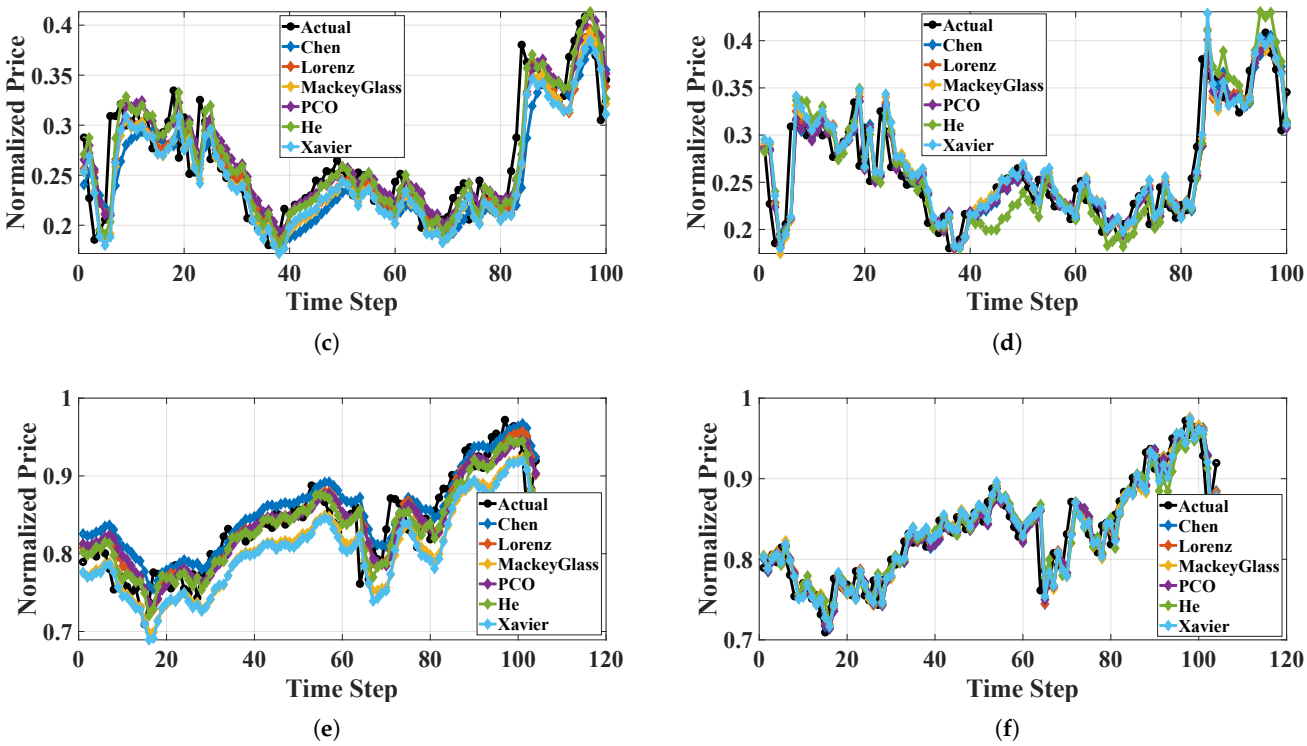

Figure 8. The proposed models' predictions overlaid on the actual data with all chaotic systems: (**a**) GRU for USD/JPY dataset, (**b**) ESN for USD/JPY dataset, (**c**) GRU for Pci-Suntek dataset, (**d**) ESN for Pci-Suntek dataset, (**e**) GRU for FTSE 100 dataset, and (**f**) ESN for FTSE 100 dataset.

6. Hardware Implementation of the Proposed ESN Model

6.1. Precision Analysis

The fixed-point implementation of the ESN prediction process in MATLAB is achieved using the '*fimath*' function. The floating-point representation provides accurate values, while the fixed-point representation simulates hardware behavior. This approach enables evaluation of numerical accuracy under realistic hardware constraints. The fractional bit width is selected based on error analysis between the floating-point reference and fixed-point implementation. As illustrated in Figure 9, a 16-bit fractional precision provides the lowest error across the tested range of 5 to 20 fractional bits.

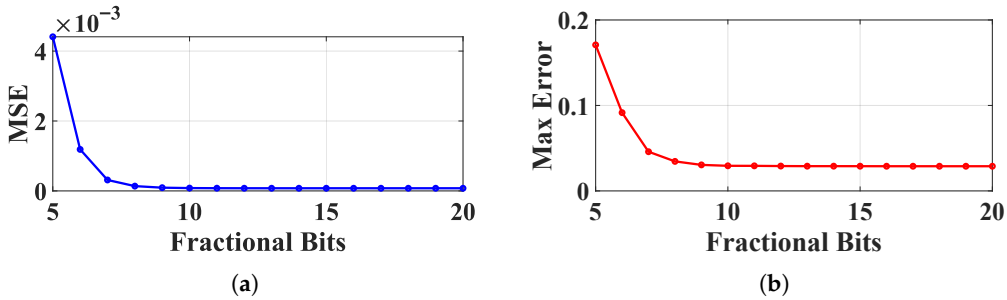

Figure 9. Error comparison between floating-point, represented by the built-in function '*tanh*' in MATLAB, and fixed-point approximation: (**a**) MSE for different fractional bits and (**b**) max error for different fractional bits.

6.2. Hyperbolic Tangent Function Implementation

The hyperbolic tangent activation function (*tanh*) in the ESN model is implemented using a polynomial approximation of order five, as shown in Figure 10. The maximum

error between the fixed-point implementation and MATLAB's built-in function is 0.028879. This polynomial approximation achieves high-accuracy and low-complexity approximation using Horner's method, as shown in the following equations:

$$\begin{aligned} Output &= C_0 + C_1 y + C_2 y^2 + C_3 y^3 + C_4 y^4 + C_5 y^5 \\ &= C_0 + y(C_1 + C_2 y + C_3 y^2 + C_4 y^3 + C_5 y^4) \\ &= C_0 + y(C_1 + y(C_2 + C_3 y + C_4 y^2 + C_5 y^3)) \\ &= C_0 + y(C_1 + y(C_2 + y(C_3 + C_4 y + C_5 y^2))) \\ &= C_0 + y(C_1 + y(C_2 + y(C_3 + y(C_4 + C_5 y)))), \end{aligned} \tag{18}$$

where y is the input signal and C_i refers to the polynomial coefficients. These coefficients are calculated using the built-in function '$polyfit$' in MATLAB, as shown in Table 3. These coefficients represent the positive region at $y \geq 0$, with the other region ($y < 0$) derived from the symmetric property:

$$\tanh(-y) = -\tanh(y). \tag{19}$$

The implementation of the *tanh* function is applied element-wise. The hardware architecture of the tanh implementation consists of only one multiplier and one adder, as shown in Figure 11. The timing diagram in Figure 12 of the '*tanh*' module shows two complete computation cycles for inputs $X1$ and $X2$, demonstrating the pipelined nature of the hyperbolic tangent calculations. The control signal '*clock*' is the positive edge clock, *Reset_n* is the negative edge reset, and the output flag is *Done*. The controller state '*state*' has values from 0 to 4. The output values from '*tanh_out*' are $Y1$ and $Y2$. The vertical dashed lines mark the clock edges, and the arrows indicate the computation latency, which is five cycles. The labels represent the computation phases. The input $X1$ is latched in the first cycle (state 0), and the polynomial is evaluated through states 1 to 4. The first output $Y1$ is carried out at the fifth cycle with output signal '$Done = 1$' at state 4. The second input $X2$ is latched in the sixth cycle in state 0.

Table 3. The polynomial coefficients for *tanh* functions using 'polyval'.

C_i	C_0	C_1	C_2	C_3	C_4	C_5
Coefficients	−0.0291416	1.2794125	−0.630065	0.1524903	−0.018083	0.0008401

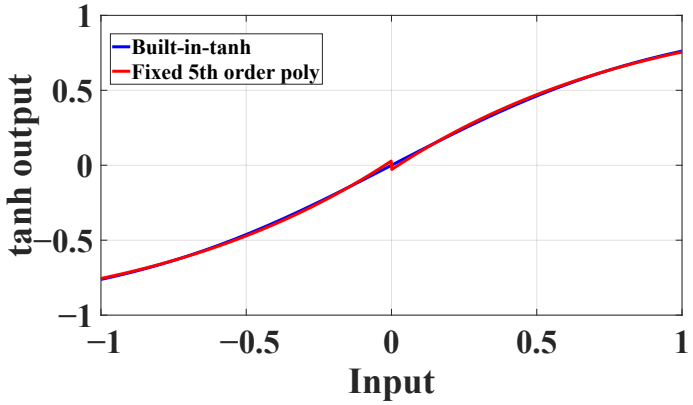

Figure 10. The *tanh* function comparison between the floating built-in function and the fixed-point of the 5th-order polynomial approximation.

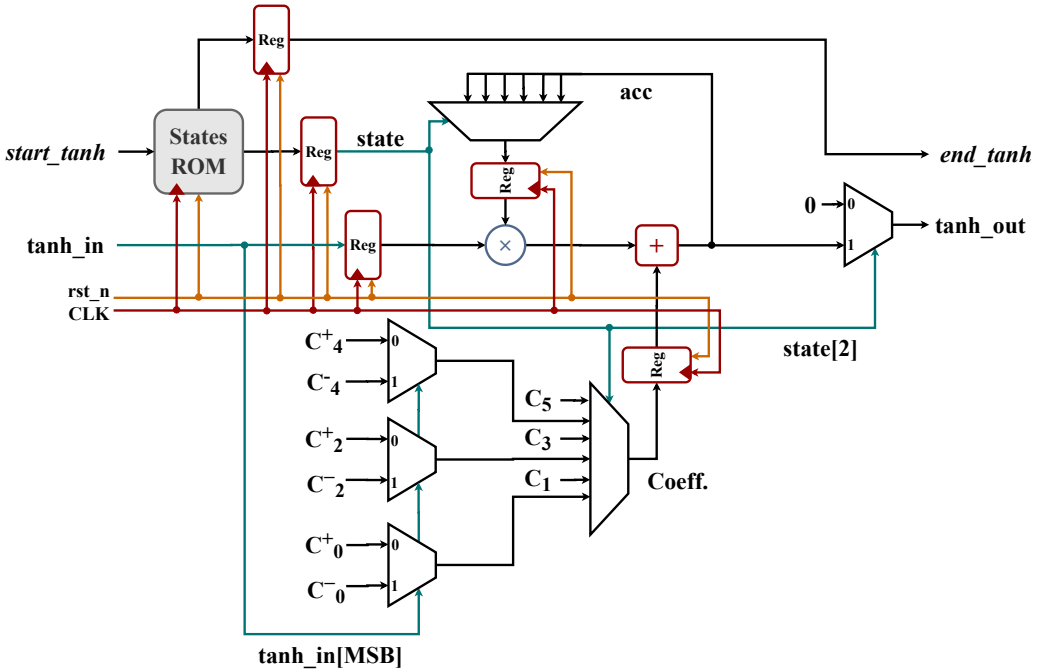

Figure 11. The polynomial approximation of the hardware implementation of the '*tanh*' function.

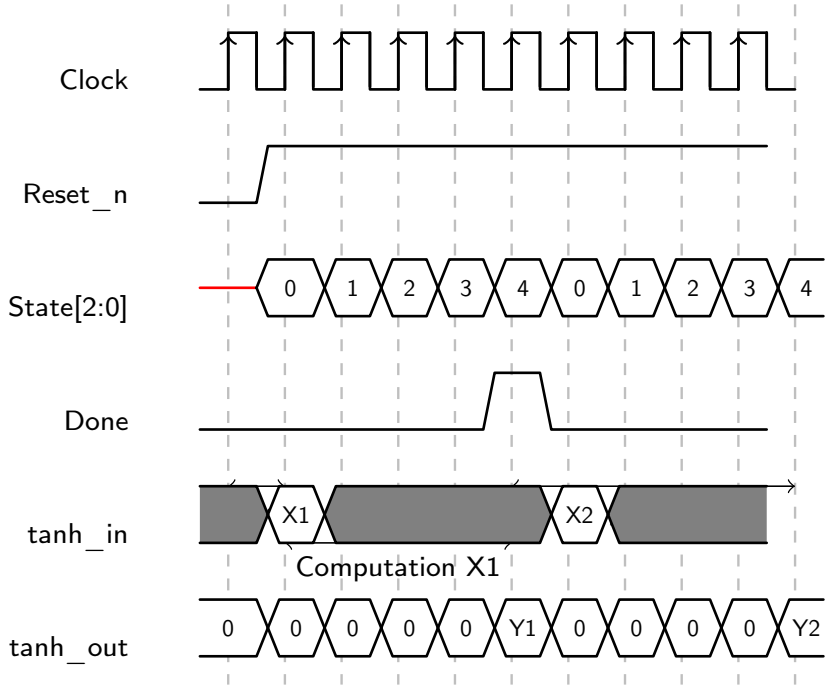

Figure 12. Timing diagram of the pipelined hyperbolic tangent (*tanh*) function module with two complete computation cycles.

6.3. Matrix Multiplication Implementation

Multiplication operations are computationally intensive and dominate resource utilization. The only two multiplication operations are the state prediction 'S_pred' and the output prediction 'Y_pred' in Equations (11) and (14). The multiplication implementation for two 100-element vectors is described by the following equation:

$$Z = \sum_{i=0}^{99} (Y1[i] \times Y2[i]), \tag{20}$$

where $Y1$ is the first input with size 1×100, and $Y2$ is the second input with size 100×1. The index i iterates from 0 to 99. The best implementation to reduce hardware complexity is to use one shared multiplication block instead of two. This design avoids duplication and reduces overall resource consumption by ∼50% compared to the dual-block design. The hardware block in Figure 13 uses 100 multipliers each for a bit-width of 19 bits.

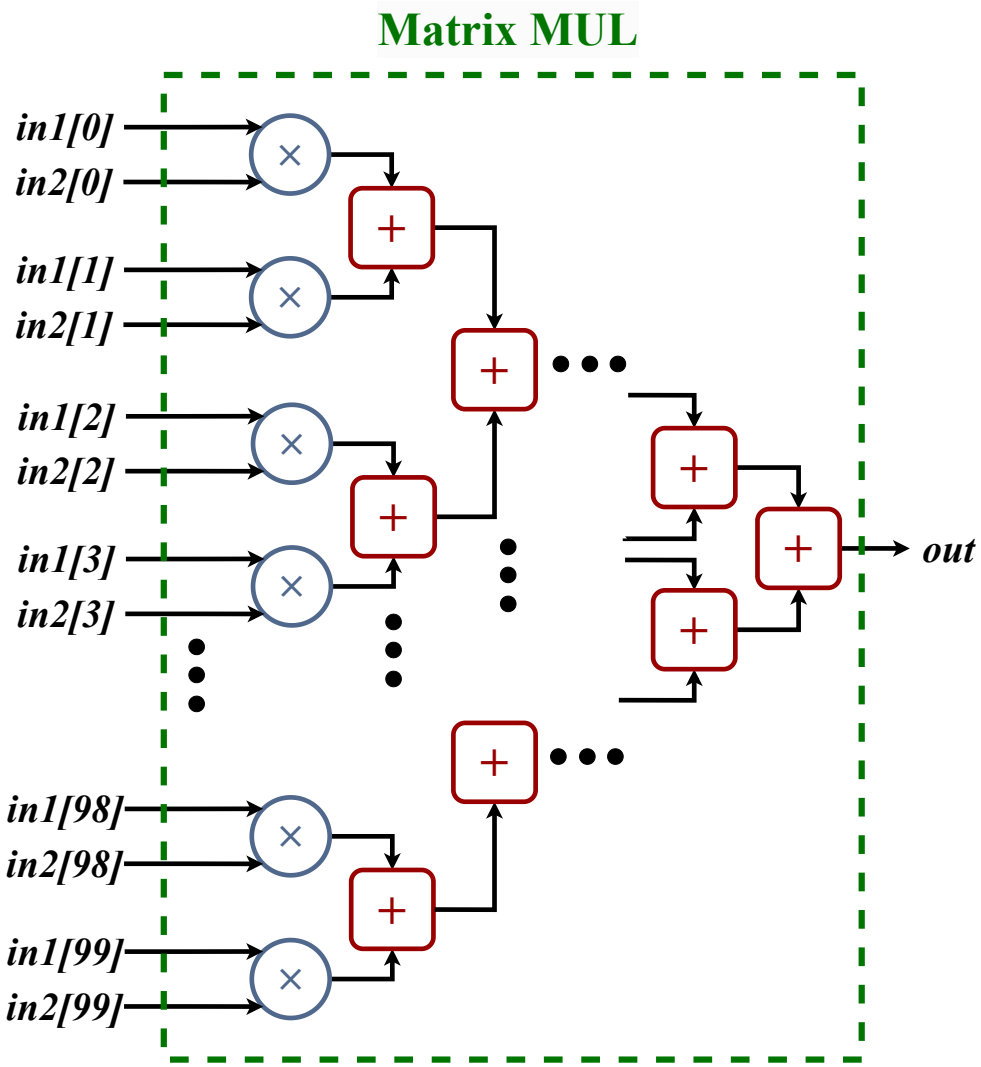

Figure 13. Matrix multiplication block of size 100.

6.4. Reservoir State Implementation

Figure 14 illustrates the top module of the proposed ESN for the updated state $S(t)$ and the predicted output $Y_{pred}(t)$ calculations. The weights W_{in}, W_{res}, and W_{out} and the previous state S_{t-1} are obtained from the software training process and then stored in memory blocks (RAMs). The size of W_{in} is 100×1, W_{res} is 100×100, W_{out} is 101×1, and $S(t-1)$ is 100×1. The address of the RAM for W_{in} and W_{res} accumulates when the signal '*tanh_done*' is activated. The shared block 'Matrix_MU' calculates the matrix multiplication operations of size 100 for the updated and prediction states. The 8-bit counter 'Counter_100' counts from 0 to 99 for each neuron index. This counter manages the state update and the output prediction operations, generating the control signal '*end_S_pred*' after reaching the value 99.

The control unit of the top proposed model in Figure 15 consists of three states. The '$IDLE$' state indicates the end of the prediction process at '$END_prediction = 1$'.

The '*S_PRED*' state computes the new reservoir state $S(t)$. The prediction output state '*Y_PRED*' computes the final predicted output when the signal '*end_S*' is activated.

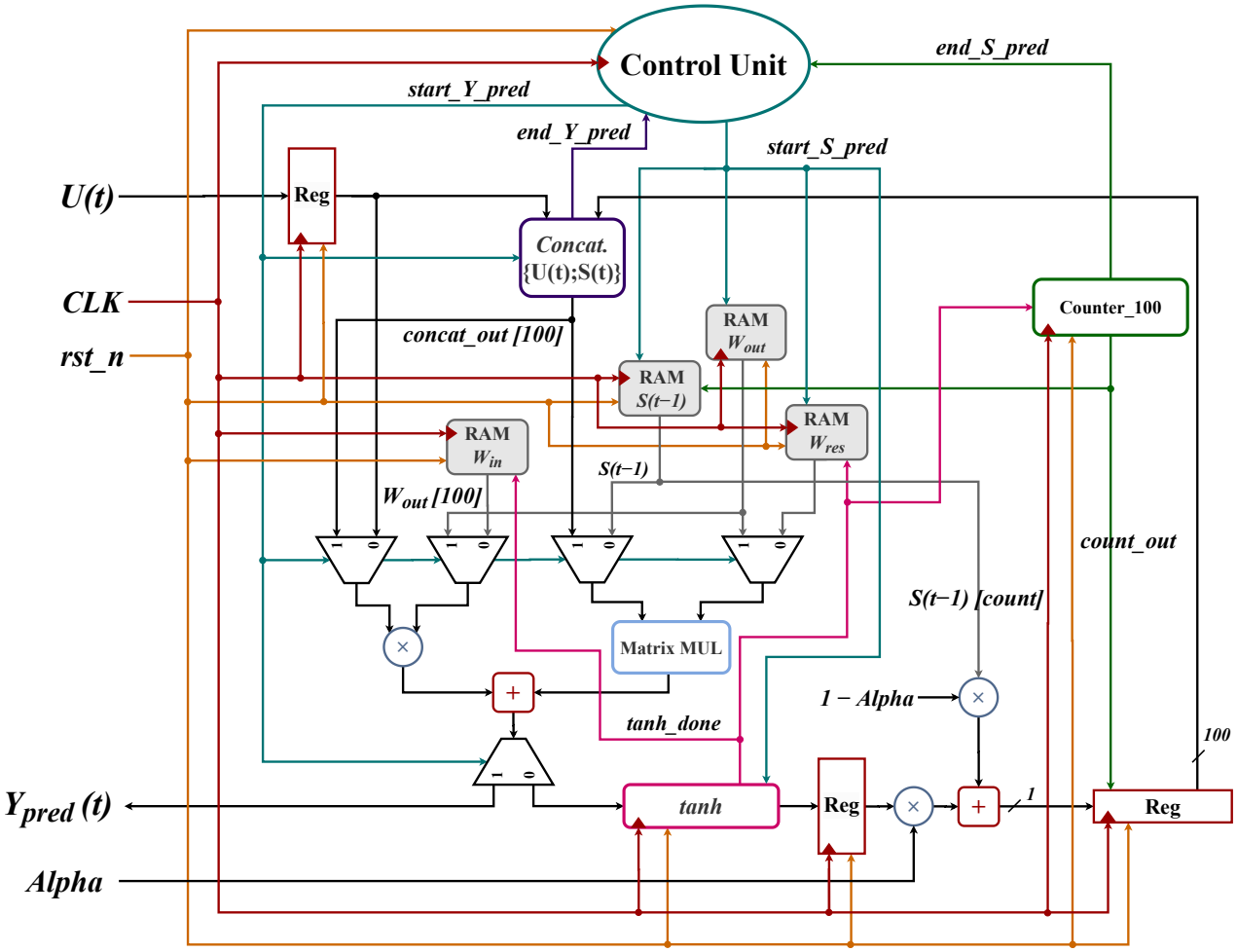

Figure 14. Top-module hardware architecture of the proposed ESN for time series forecasting prediction.

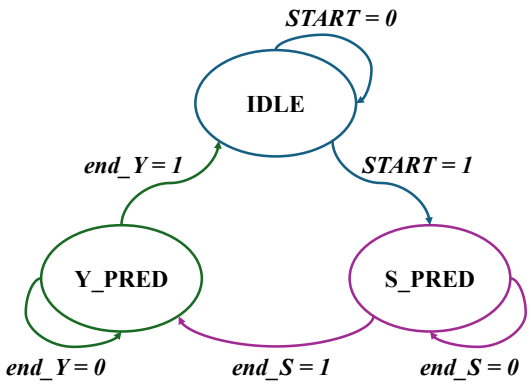

Figure 15. Control unit of the proposed top-module ESN.

6.5. Utilization Results and Performance Evaluation Comparison

The architecture of the proposed ESN model is implemented on the Kintex-UltraScale KCU105 FPGA board, achieving a maximum frequency of 83.5 MHz. The total power consumption of the design equals 0.677 W, with a dynamic power of 0.197 W. The total prediction latency is 496 cycles, corresponding to 4.96 µs. Table 4 presents the resource utilization of the design, which includes the look-up table (LUT), block RAM (BRAM), flip-flop (FF), and digital signal processing block (DSP). The DSP utilization is 5.42% (104 units)

and the BRAM utilization is 4.75% (28.5 blocks), demonstrating an efficient mapping of reservoir computations to FPGA hardware. The low LUTs/FFs utilization confirms minimal overhead for control logic, highlighting the efficiency for latency-critical edge inference.

Table 4. The resource utilization on FPGA of the proposed ESN.

Resource	Utilization	Available	Utilization %
LUTs	15,833	2,424,000	6.53%
FFs	4306	484,800	0.89%
BRAMs	28.50	600	4.75%
DSPs	104	1920	5.42%

Table 5 states that the proposed ESN achieves lower power than the models in [18]. The performance calculations are given by

$$\text{Performance } (GOPS) = \text{N_operations} \times F_{max} \times 10^{-9}, \tag{21}$$

where $GOPS$ is the Giga Operations Per Second, and F_{max} is the maximum operating frequency. The number of operations is 102 per cycle, indicating the maximum number of adders and multipliers that operate simultaneously. The performance is 8.517 $GOPS$, so the efficiency in terms of $GOPS/W$ is 12.58.

Table 5. Hardware implementation results between the proposed ESN model and the other ESN models in [18].

Metric	ESN in [18]			intESN in [18]			ESN (Proposed)
Neurons	100	200	300	100	200	300	100
Device	Xilinx Zynq-7000 FPGA			Xilinx Zynq-7000 FPGA			Kintex-UltraScale KCU105
Max Frequency (MHz)	100 MHz			100 MHz			83.5 MHz
Power (W)	1.73	1.78	1.95	1.59	1.6	1.6	0.677 W (dynamical power 0.197 W)
Precision	32-bit float			3-bit integer			19-bit (16-bit fractional, 3-bit integer)

7. Conclusions

This study examined the efficacy of chaotic dynamical systems in initializing the weights and biases of neural network models, including GRU and ESN. This work presented a hardware accelerator ESN model with Lorenz chaos training for accurate time series prediction. This proposed model was compared with the proposed GRU model, both with and without chaotic integration, to validate the prediction accuracy. The ESN and GRU models were trained by four different chaotic systems, including the standard Chen system, PCO, Lorenz system, and Mackey–Glass equation, and two standard systems, named He and Xavier initialization. The validation of the models was implemented using six different datasets: S&P 500, ISEQ overall historical data, XAU/USD, USD/JPY, Pci-Suntek Tech stock, and FTSE 100 historical data. The prediction results of the proposed ESN model in hardware simulation were compared to those of floating and fixed-point representations. The proposed ESN model, trained by the Lorenz system, delivered the lowest prediction errors. The hardware implementation of the proposed ESN architecture achieved a power consumption of 0.677 W at a maximum frequency of 83.5 MHz on Kintex UltraScale KCU105 FPGA board. Future work will explore different models, such as transformer and autoencoder, for more accuracy and generalization in financial time series prediction tasks. Additionally, we plan to explore different time horizons to achieve a more efficient and stable model.

Author Contributions: Conceptualization, Z.A.H., M.H.Y., and L.A.S.; methodology, Z.A.H., M.H.Y., and L.A.S.; software, Z.A.H. and M.H.Y.; validation, Z.A.H. and M.H.Y.; formal analysis, Z.A.H. and M.H.Y.; investigation, Z.A.H. and M.H.Y.; resources, L.A.S.; data curation, Z.A.H. and M.H.Y.; writing—original draft preparation, Z.A.H.; writing—review and editing, M.H.Y. and L.A.S.; visualization, Z.A.H. and M.H.Y.; supervision, L.A.S.; project administration, L.A.S.; funding acquisition, L.A.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research is funded by the Arab-German Young Academy of Sciences and Humanities (AGYA). AGYA promotes sustainable research cooperation between outstanding Arab and German scholars (3–10 years post-PhD) from all disciplines. AGYA draws on financial support of the German Federal Ministry of Research, Technology, and Space (BMFTR): grant 01DL25001. The authors remain solely responsible for the content and recommendations provided in this publication, which do not reflect the positions of AGYA or any of its funding partners.

Data Availability Statement: S&P 500 dataset: https://www.spglobal.com/spdji/en/indices/equity/sp-500 (accessed on 19 July 2025). ISEQ dataset: https://finance.yahoo.com/quote/%5EISEQ/history/ (accessed on 19 July 2025). XAU/USD, USD/JPY, Pci-Suntek, and FTSE 100 datasets: https://finance.yahoo.com/ (accessed on 19 July 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Wang, H.; Mo, Y. Sparse compressed deep echo state network with improved arithmetic optimization algorithm for chaotic time series prediction. *Expert Syst. Appl.* **2025**, *259*, 125249. [CrossRef]
2. Chen, J.; Wen, Y.; Nanehkaran, Y.; Suzauddola, M.; Chen, W.; Zhang, D. Machine learning techniques for stock price prediction and graphic signal recognition. *Eng. Appl. Artif. Intell.* **2023**, *121*, 106038. [CrossRef]
3. Wen, S.C.; Yang, C.H. Time series analysis and prediction of nonlinear systems with ensemble learning framework applied to deep learning neural networks. *Inf. Sci.* **2021**, *572*, 167–181. [CrossRef]
4. Yu, H.; Dong, J.; Li, B. Economic insight through an optimized deep learning model for stock market prediction regarding Dow Jones industrial Average index. *Expert Syst. Appl.* **2025**, *291*, 128473. [CrossRef]
5. Huang, Y.; Pei, Z.; Yan, J.; Zhou, C.; Lu, X. A combined Adaptive Gaussian Short-Term Fourier Transform and Mamba framework for stock price prediction. *Eng. Appl. Artif. Intell.* **2025**, *162*, 112588. [CrossRef]
6. Jia, B.; Wu, H.; Guo, K. Chaos theory meets deep learning: A new approach to time series forecasting. *Expert Syst. Appl.* **2024**, *255*, 124533. [CrossRef]
7. Viehweg, J.; Walther, D.; Mäder, P. Temporal convolution derived multi-layered reservoir computing. *Neurocomputing* **2025**, *617*, 128938. [CrossRef]
8. Pham, P.; Pedrycz, W.; Vo, B. Dual attention-based sequential auto-encoder for COVID-19 outbreak forecasting: A case study in Vietnam. *Expert Syst. Appl.* **2022**, *203*, 117514. [CrossRef] [PubMed]
9. Yan, M.; Huang, C.; Bienstman, P.; Tino, P.; Lin, W.; Sun, J. Emerging opportunities and challenges for the future of reservoir computing. *Nat. Commun.* **2024**, *15*, 2056. [CrossRef]
10. Bukhari, A.H.; Raja, M.A.Z.; Alquhayz, H.; Almazah, M.M.; Abdalla, M.Z.; Hassan, M.; Shoaib, M. Predictive analysis of stochastic stock pattern utilizing fractional order dynamics and heteroscedastic with a radial neural network framework. *Eng. Appl. Artif. Intell.* **2024**, *135*, 108687. [CrossRef]
11. Zhou, X.; Hao, Y.; Liu, Y.; Dang, L.; Qiao, B.; Zuo, X. Short-term prediction of dissolved oxygen and water temperature using deep learning with dual proportional-integral-derivative error corrector in pond culture. *Eng. Appl. Artif. Intell.* **2025**, *142*, 109964. [CrossRef]
12. Chattopadhyay, A.; Hassanzadeh, P.; Subramanian, D. Data-driven predictions of a multiscale Lorenz 96 chaotic system using machine-learning methods: Reservoir computing, artificial neural network, and long short-term memory network. *Nonlinear Processes Geophys.* **2020**, *27*, 373–389. [CrossRef]
13. Gonbadi, L.; Rostami, H.; Sahafizadeh, E.; Rostami, S.; Nejad, M.M.; Shirzadi, A. Input driven optimization of echo state network parameters for prediction on chaotic time series. *Sci. Rep.* **2025**, *15*, 33005. [CrossRef]
14. Chen, X.; Chen, L.; Li, S.; Jin, L. A mirrored echo state network with application to time series prediction. *Inf. Sci.* **2025**, *716*, 122260. [CrossRef]
15. Wang, H.; Wang, Z.; Yu, M.; Liang, J.; Peng, J.; Wang, Y. Grouped Vector Autoregression Reservoir Computing Based on Randomly Distributed Embedding for Multistep-Ahead Prediction. *IEEE Trans. Neural Netw. Learn. Syst.* **2025**, *36*, 17265–17279. [CrossRef]

16. Na, X.; Ren, W.; Liu, M.; Han, M. Hierarchical Echo State Network With Sparse Learning: A Method for Multidimensional Chaotic Time Series Prediction. *IEEE Trans. Neural Netw. Learn. Syst.* **2023**, *34*, 9302–9313. [CrossRef] [PubMed]
17. Tang, Y.; Song, Z.; Zhu, Y.; Hou, M.; Tang, C.; Ji, J. Adopting a dendritic neural model for predicting stock price index movement. *Expert Syst. Appl.* **2022**, *205*, 117637. [CrossRef]
18. Kleyko, D.; Frady, E.P.; Kheffache, M.; Osipov, E. Integer Echo State Networks: Efficient Reservoir Computing for Digital Hardware. *IEEE Trans. Neural Netw. Learn. Syst.* **2022**, *33*, 1688–1701. [CrossRef] [PubMed]
19. Hassaan, Z.A.; Yacoub, M.H.; Said, L.A. Gated recurrent unit accelerators for financial time series prediction on field-programmable gate array. *Eng. Appl. Artif. Intell.* **2025**, *162*, 112534. [CrossRef]
20. Sharobim, B.K.; Yacoub, M.H.; Sayed, W.S.; Radwan, A.G.; Said, L.A. Artificial Neural Network Chaotic PRNG and simple encryption on FPGA. *Eng. Appl. Artif. Intell.* **2023**, *126*, 106888. [CrossRef]
21. Mao, G.; Rahman, T.; Maheshwari, S.; Pattison, B.; Shao, Z.; Shafik, R.; Yakovlev, A. Dynamic Tsetlin Machine Accelerators for On-Chip Training Using FPGAs. *IEEE Trans. Circuits Syst. I Regul. Pap.* **2025**, *72*, 6962–6975. [CrossRef]
22. Wasef, M.; Rafla, N. SoC Reconfigurable Architecture for Implementing Software Trained Recurrent Neural Networks on FPGA. *IEEE Trans. Circuits Syst. I Regul. Pap.* **2023**, *70*, 2497–2510. [CrossRef]
23. Liu, F.; Li, H.; Hu, W.; He, Y. Review of neural network model acceleration techniques based on FPGA platforms. *Neurocomputing* **2024**, *610*, 128511. [CrossRef]
24. *Foreign Currency Financial Reporting from Euro to Yen to Yuan*; Wiley: Hoboken, NJ, USA, 2012; pp. 251–266. [CrossRef]
25. Zhang, X.; Zhu, Y.; Lou, X. Reconfigurable and Energy-Efficient Architecture for Deploying Multi-Layer RNNs on FPGA. *IEEE Trans. Circuits Syst. I Regul. Pap.* **2024**, *71*, 5969–5982. [CrossRef]
26. AbdElbaky, M.H.; Yacoub, M.H.; Sayed, W.S.; Said, L.A. High-performance FPGA-accelerated LSTM neural network for chaotic time series prediction. *AEU Int. J. Electron. Commun.* **2025**, *199*, 155845. [CrossRef]
27. Xu, H.; Chai, L.; Luo, Z.; Li, S. Stock movement prediction via gated recurrent unit network based on reinforcement learning with incorporated attention mechanisms. *Neurocomputing* **2022**, *467*, 214–228. [CrossRef]
28. Farhadi, A.; Zamanifar, A.; Alipour, A.; Taheri, A.; Asadolahi, M. A Hybrid LSTM-GRU Model for Stock Price Prediction. *IEEE Access* **2025**, *13*, 117594–117618. [CrossRef]
29. Li, T.; Guo, Z.; Li, Q. Decomposition-based deep projection-encoding echo state network for multi-scale and multi-step wind speed prediction. *Expert Syst. Appl.* **2025**, *266*, 126074. [CrossRef]
30. Gong, Y.; Lun, S.; Li, M. Broad-ESN Based on Radical Activation Function for Predicting Time Series With Multiple Variables. *IEEE Trans. Neural Netw. Learn. Syst.* **2025**, *36*, 17310–17321. [CrossRef]
31. Delshad, A.; Cherry, E.M. Predicting complex time series with deep echo state networks. *Chaos* **2025**, *35*, 093126. [CrossRef]
32. Sharifi Ghazijahani, M.; Cierpka, C. On the spatial prediction of the turbulent flow behind an array of cylinders via echo state networks. *Eng. Appl. Artif. Intell.* **2025**, *144*, 110079. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.